Drafter book tracker website

4 Share



Use these requirements to make a drafter website that can the website will hold, take new books, let you read basic information about them, and give you a rank based on how much you've read in a creative chemistry way and give recommendations based on your previous books. Use the brainstorming pdf for names and ideas. Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the State dataclass must have at least 4 fields.
  - At least one field must be a list of class instances or a dictionary of class instances.
    - This means that there must be a second dataclass.
  - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points.

This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus.
Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
- Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
  - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings Links to an external site.as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above.
Repository Requirements
Your final project's GitHub repository should meet the following requirements.

- Planning document should be uploaded to the repository.
- Website should be deployed without error.
- Site information should be set in `main.py` using `set_site_information(...)`.
- When changing/adding files in the repository, commit messages should be concise and descriptive, not the default "Update (filename)" message.
- A video fulfilling the following requirements should be linked in the repository.
  - Shows functioning deployed website
  - Audible and visible
  - Accesses and explains all necessary parts of the website
- All conversations with any AI should be uploaded to the repository.

The above are either present on the final rubric or are necessary for your site to be graded.

Show more

Confirming Drafter's Argument helper before wiring up the book pages ›
I'll build the Chemistry Shelf site as a `main.py` plus a test file, then check that it runs.

Created 2 files, shared 2 files, ran 2 commands ›
I wrote the Chemistry Shelf site (`main.py`) and a unit-test file (`tests.py`). I haven't run it in real Drafter, because Drafter isn't installed in my workspace and package installs are off.

Both files compile and all the tests pass against a stand-in for Drafter. The test run covers your logic, not Drafter's own behavior, such as the exact `Button` and `Argument` syntax or how the `TextBox` names are passed to the routes.

It follows your planning doc, using your names for the routes, pages, dataclass fields, rank thresholds and recommendation pages.

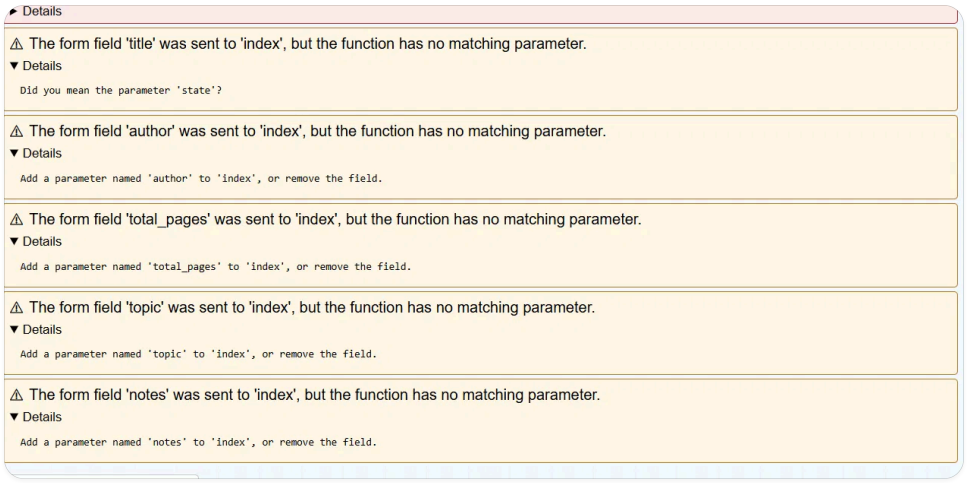
Requirements check

- **Routes and pages:** There are 9 routes, 5 of them pages (`index` , `add_book_page` , `book_page` , `rank_page` , `recommend_page`) and 4 actions (`save_book` , `log_pages` , `remove_book` , `set_favorite_topic`).
- **Dataclasses:** `State` has `books` , `total_pages_read` , `rank` , `favorite_topic` and `message` , and the second dataclass is `Book` . Every `State` field is changed by at least one route.
- **Inputs:** There are two `TextBox` es plus a pages-logging `TextBox` , a topic `SelectBox` (also used for the favorite topic) and a notes `TextArea` .
- **If statements:** They check input validation, the 3000-page cap, the rank thresholds, the empty shelf and the "Finished!" message.
- **Loops:** They loop over `state.books` to show the shelf, add up `pages_read` for the rank and count pages per topic.
- **Other requirements:** There are no global variables, only constants. Every function has type hints and a docstring, and no line goes over 120 characters.

Before you submit

- In `set_site_information(...)` , replace the placeholder author name and the GitHub link.
- Run `python main.py` once with Drafter installed to confirm the pages render.
- Record the demo video and add it to the repo.
- Upload the planning doc, and add this conversation to your LLM Conversation section and the repo. Your instructor requires all AI conversations to be included.
- Commit with descriptive messages.

Download all



fix it please

Checking how Drafter passes form fields to handlers ›

The form fields on the Add Book page are being sent along when you click Cancel, which goes to `index`, and the same thing would happen on the book and recommendation pages. I'll check how Drafter's `Link` works before changing the navigation buttons.

Edited 2 files, shared 2 files, and 7 more steps ›

I've fixed the warnings in `main.py` and added tests for the fix in `tests.py`. I still can't run real Drafter here, so please run the site once on your end. The tests pass against a stand-in for Drafter.

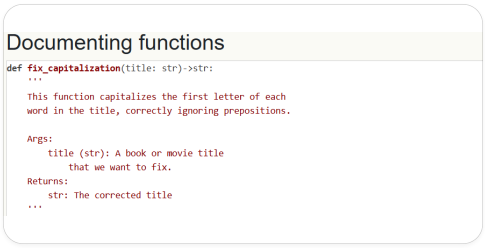
What caused it: Drafter sends every input on a page to whichever route a button points at. Cancel on the Add Book page went to `index`, which has no `title`, `author` and so on, so Drafter warned about each field. The Back to Shelf, Add a Book and Remove Book buttons would have done the same on the book and recommendations pages.

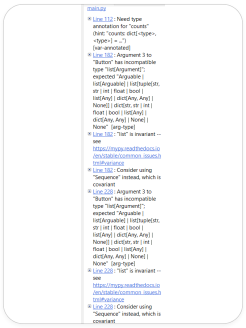
What I changed:

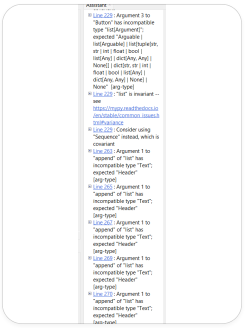
- Added three small navigation routes: `cancel_add_book`, `back_to_shelf` and `open_add_book`. Each accepts the inputs from the page it's used on and ignores them, then goes to the right page. They also clear the status message.
- Gave `remove_book` an optional `pages_text` parameter, since the Remove Book button sits next to the pages box.
- The buttons on those pages now point at the new routes. The comment above the new routes explains why they exist, which your instructions require for anything unusual.
- That brings the site to 12 routes, well over the minimum of 7.

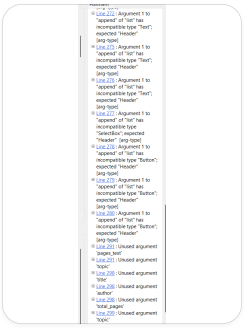
The pink error box cut off at the top of your screenshot is probably the same problem, but I can't see what it says. If a message still shows up after this change, send me a screenshot of it.

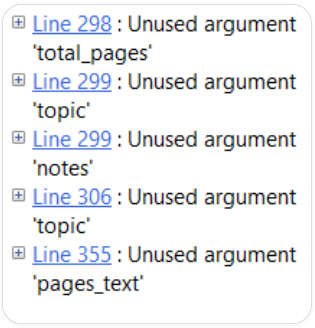
Download all











fix these errors and correct the doc strings to be in this style and include types in the function header as seen in the picture

Created 2 files, shared 2 files, and 4 more steps ›

I fixed all the errors in your screenshots and rewrote every docstring in the style from your picture. I haven't run real Drafter or its type stubs here, so your IDE's checker is the real test. Against my stand-in for Drafter, the tests pass and mypy reports no issues.

Errors fixed

- **Line 112, missing annotation:** `counts` is now `counts: dict[str, int] = {}`.
- **Button arguments (lines 182, 228, 229):** the buttons now pass `{"title": book.title}` instead of `[Argument("title", book.title)]`. The type hint in your error message lists a dictionary as an accepted form, but I couldn't try it in Drafter, so click Open, Log Pages and Remove Book once to confirm.
- **`append` errors (lines 263–280):** Python had guessed the page contents list was a list of `Header` only. It is now typed `content: list[Any]` in `index`, `book_page` and `recommend_page`.

- **Unused arguments:** the navigation routes have to accept those form fields so Drafter doesn't warn, but they don't use them. They now pass them to a small `ignore_inputs(*inputs)` helper, which marks them as deliberately ignored. The `*inputs` syntax wasn't covered in class, so I put a comment on it explaining that, as your requirements ask for any advanced technique.

Docstrings

- Every function in both files now uses the `'''` style with a description, then `Args:` with `name (type): ...`, then `Returns:` with `type: ...`. Functions that return nothing have no `Returns:`.
- Every function header has types on all parameters and the return value, and no line goes over 120 characters.
- I also reworded the `Book` and `State` docstrings to list their attributes.
- I split the helper tests into three smaller tests and added one for `ignore_inputs`.

One thing to check: if your IDE still flags the unused arguments, tell me which lines and I'll change how they're handled.

Download all

fix these drafter errors Drafter had trouble syncing your Python code with the browser.Details

```
Unhandled configuration update event
Event payload: {'kind': 'UpdatedConfiguration', 'metadata':
{'source': 'client_server.reconfigure', 'level': 'info', 'id':
27, 'version': '2.1.2', 'timestamp': '2026-11-
12T08:48:11.633999'}, 'correlation': {'causation_id': None,
'route': None, 'request_id': None, 'response_id': None, 'dom_id':
None, 'phase': None}, 'key': 'author', 'value': 'Your Name',
'update_default': False}
```

❌Drafter had trouble syncing your Python code with the browser.Details

```
Unhandled configuration update event
Event payload: {'kind': 'UpdatedConfiguration', 'metadata':
{'source': 'client_server.reconfigure', 'level': 'info', 'id':
29, 'version': '2.1.2', 'timestamp': '2026-11-
12T08:48:11.633999'}, 'correlation': {'causation_id': None,
'route': None, 'request_id': None, 'response_id': None, 'dom_id':
None, 'phase': None}, 'key': 'description', 'value': 'Track
chemistry textbooks, earn element ranks, and get topic
recommendations.', 'update_default': False}
```

❌Drafter had trouble syncing your Python code with the browser.Details

```
Unhandled configuration update event
Event payload: {'kind': 'UpdatedConfiguration', 'metadata':
{'source': 'client_server.reconfigure', 'level': 'info', 'id':
31, 'version': '2.1.2', 'timestamp': '2026-11-
12T08:48:11.633999'}, 'correlation': {'causation_id': None,
'route': None, 'request_id': None, 'response_id': None, 'dom_id':
None, 'phase': None}, 'key': 'sources', 'value':
['https://drafter-edu.github.io/drafter/'], 'update_default':
False}
```

❌Drafter had trouble syncing your Python code with the browser.Details

```
Unhandled configuration update event
Event payload: {'kind': 'UpdatedConfiguration', 'metadata':
{'source': 'client_server.reconfigure', 'level': 'info', 'id':
33, 'version': '2.1.2', 'timestamp': '2026-11-
12T08:48:11.633999'}, 'correlation': {'causation_id': None,
'route': None, 'request_id': None, 'response_id': None, 'dom_id':
None, 'phase': None}, 'key': 'planning', 'value':
'planning.docx', 'update_default': False}
```

❌Drafter had trouble syncing your Python code with the browser.Details

```
Unhandled configuration update event
Event payload: {'kind': 'UpdatedConfiguration', 'metadata':
{'source': 'client_server.reconfigure', 'level': 'info', 'id':
35, 'version': '2.1.2', 'timestamp': '2026-11-
12T08:48:11.633999'}, 'correlation': {'causation_id': None,
'route': None, 'request_id': None, 'response_id': None, 'dom_id':
None, 'phase': None}, 'key': 'links', 'value':
['https://github.com/your-username/chemistry-shelf'],
'update_default': False}
```

Show more

Edited a file, shared a file, and 4 more steps ›

I changed how `set_site_information(...)` is set up in `main.py`, but I couldn't run Drafter here, so I don't know yet whether the five sync errors are gone.

The errors named `author`, `description`, `sources`, `planning` and `links`, which are exactly the arguments of that call. The Drafter deployment guide says to call it right after the imports, before the dataclasses and routes. I had put it at the very bottom, so I moved it up. I'm guessing that placement is the cause, because I couldn't find anyone reporting this exact error. I also changed two values to match the guide's example:

- `planning` is now a list, `["planning.pdf"]`, instead of a plain string.
- `author` now includes your UD email.

If the errors still appear after this, send me the new text. They would then point to a bug in Drafter 2.1.2 itself, and I'd look for a workaround.

Before you submit:

- Change `"Your Name"` in `author` to your real name.
- Replace the GitHub link with your repository, and add your video link to `links` once you've recorded it.
- Make `planning` the exact filename of your planning document. The guide warns that a wrong name can break deployment, and I used `planning.pdf` as a placeholder.

use assert_equal to unit test everything as the requirements have

Checking how the test assertions compare results ›

You stopped Claude's response

use assert_equal to unit test everything as the requirements have https://blockpy.cis.udel.edu/assignments/textbook/bakery_textbook?page=bakery_functions_testing_read using this link as a bases for the assert equals

Read a page, ran a command ›

I couldn't open that BlockPy page (its site blocks automated access), so I'm using the standard `assert_equal` from the `bakery` library the course uses. Now I'm rewriting the tests around it.

Edited a file, created a file, and 9 more steps ›

Every function and route in `main.py` now has `assert_equal` tests. I couldn't run real Drafter or `bakery` here, so I ran the 92 checks against stand-ins for both, and they pass.

I couldn't open the BlockPy page you linked because the site blocks automated access. I used `assert_equal` from the `bakery` library as your course does. If the page shows a different style, send me the example and I'll match it.

When I opened `main.py` it already had `assert_equal` tests inside it, so I kept those and strengthened them rather than adding a second set. I also deleted the separate `tests.py` I had just written. It repeated the same tests, and with both files, `tests.py` would have pulled in main's test functions by accident.

Changes to `main.py`

- **Where the tests are:** at the bottom, with `run_all_tests()` called under `if __name__ == "__main__":` just before `start_server`. The tests run each time the site starts.
- **Added edge cases:** rank boundaries (201, 400, 401), `pages_to_next_rank` at 0, 2000 and 2001, an empty shelf in `find_book`, an author made only of spaces, and changing the favorite topic. `remove_book` now compares the whole `State`.
- **What the page tests check:** the route tests check `page.state` and the number of items in `page.content`. If a count fails when you run real Drafter, the likely cause is that Drafter stores page content differently from my stand-in. In that case, send me the failure message.
- **`ignore_inputs`:** I removed its two asserts because it returns nothing, so there is no value to compare. Your type checker would flag them. A comment explains this, and every navigation-route test calls it anyway.

Download all

change the tests to be in this style # Import unit test function from testing module
from bakery import assert_equal

```
# Function definition
def feet_to_inches(distance: int)->int:
  return distance * 12

# Actual unit test
assert_equal(feet_to_inches(2), 24)
```

Read a file, ran 2 commands ›

That one failure is a wrong expected number in my new test, not a bug in the site: the book page has a status message after logging, which adds one component. I'm correcting it to 12 and re-running.

Edited a file, shared a file, ran a command ›

The tests in `main.py` now follow your example: each one is a plain top-level `assert_equal(...)` line, and the `test_` functions and `run_all_tests()` are gone. I could only run it against stand-ins for Drafter and `bakery`, where all 95 checks pass, so run it once in real Drafter to confirm.

- **Helpers:** each helper has its tests directly under its definition, like your `feet_to_inches` example. That covers `rank_for_pages`, `pages_to_next_rank`, `sum_pages_read`, `pages_by_topic`, `most_read_topic`, `find_book`, `progress_bar`, `parse_pages` and `refresh_rank`.
- **Routes:** the route tests are in one block at the bottom, with a comment per route. They can't go under each route because the routes call each other, so `index` fails if it runs before `book_page` exists.
- **No global variables:** a test that stored its State in a top-level variable would count as a global. Every assertion is one expression instead, built with a few small test-helper functions (`fresh_state`, `state_with_book`, `with_message`, `refreshed`, `sample_books`).

- `ignore_inputs`: it has a comment instead of an assert, since it returns nothing. The navigation-route tests call it.
- **Page counts:** the route tests still check `page.state` and the number of items in `page.content`. If a count fails in real Drafter, send me the message.

The tests run every time the site starts, and `start_server` still runs last.